
CSE 234 Project 2: Supervised Fine Tuning

Owen Park
owpark@ucsd.edu

Shaurya Raswan
sraswan@ucsd.edu

1 Training Data Methodology

1.1 Data Composition and Serialization

Our training corpus aggregates natural language queries and their corresponding gold standard SQL statements alongside relational database schemas. The foundational schema data was originally structured in JSON Spider format (which map columns to tables via zero-indexed arrays). But that poses a token overhead and relationship confusion for causal language models.

To optimize for context window constraints and improve sequence processing, we flattened the JSON into a simple string representation format like:

```
Table [TableName]: [Col1], [Col2], ...
```

This bounds columns to their parent tables and removes unnecessary metadata like the first empty column at index -1. We used this for both pre-training and inference, so this aligned the input closer to the natural language distributions the models observed during pre-training. We tried many schema filtering and sorting methods, however, we got the best accuracy without any filtering, possibly because our filtering methods were too noisy. In addition, we did not include SQL datatypes or keytypes.

1.2 Output

The output format was simply JSON, and we removed invalid schema outputs (table names and column names that do not exist, invalid JSON parsing, etc.).

1.3 Data Augmentation and Glossary Injection

A primary challenge in text-to-SQL tasks is the cryptic abbreviations and compressed identifiers. These conventions create a distinct semantic gap, as the underlying lexical tokens lack direct mapping to the user's natural language queries.

We tested many things, including augmenting the training inputs with a dynamic glossary mapping system. During the data preparation phase, table and column identifiers were tokenized and processed through an expansion dictionary to resolve domain-specific abbreviations into their natural language equivalents (like mapping NumRepArs to number repetitive areas). This generated glossary was injected directly into the prompt context alongside the serialized schema. This was used with the SNAILS original artifact, which was natural mapping walks as csv files. However, we were unable to get our training to get an acceptable score at the time and this method is prone to overfitting with the SNAILS dataset.

1.4 Diagnostic Metrics

We implemented a set-based evaluation pipeline that strictly measures the alignment between the predicted schema links and the ground truth. This is the same method as the evaluation. Because the model outputs structured JSON, the evaluation first passes the generated strings through a parser designed to extract the first valid JSON object, handling malformed generation if it happens.

At table level and column level, we calculate the standard classification metrics based on True Positives (TP), False Positives (FP), and False Negatives (FN).

- Precision (P): $\frac{TP}{TP+FP}$ (Penalizes the model for hallucinating non-existent tables/columns or including unnecessary distractor elements).
- Recall (R): $\frac{TP}{TP+FN}$ (Penalizes the model for missing strictly necessary tables/columns).
- F_1 Score: $2 \cdot \frac{P \cdot R}{P+R}$ (The harmonic mean of precision and recall).

These follow into three final composite scores averaged across the entire evaluation dataset:

- Table Score
- Column Score
- Leaderboard Score: $0.5 \cdot \text{Table Score} + 0.5 \cdot \text{Column Score}$, representing the model’s overall efficacy in zero-shot schema pruning and column selection.

These serve as the way we actually train during SFT, which will be explained later.

2 Final Pipeline Architecture

2.1 Final Pipeline

The core of our schema-linking pipeline utilizes the meta-llama/Llama-3.2-1B-Instruct base model. We implemented a LoRA strategy. The adapter was configured with a rank (r) of 32 and a lora_alpha of 64, targeting all major projection matrices (q_proj , k_proj , v_proj , o_proj , gate_proj , up_proj , down_proj) to maximize the model’s capacity to learn the complex lexical mappings of the database schemas. Dropout was set to 0.1, and biases were disabled.

The training was executed using SFT over 3 epochs. To maintain a manageable memory footprint while simulating a larger batch size, we utilized a per-device train batch size of 1 with 8 gradient accumulation steps. The optimizer applied a cosine learning rate scheduler peaking at $2e-4$, operated in bf16 mixed precision, and utilized gradient checkpointing. Crucially, we enabled `completion_only_loss` to mask the prompt tokens during backpropagation, forcing the model to calculate loss strictly on the generated JSON output rather than the schema context.

2.2 Prompt Template, Decoding, and Post-Processing

Inputs were formatted using a strict System/User/Assistant chat template. The system prompt explicitly defined the zero-shot schema-linking rules, restricting the model to exact table/column names and specifying the expected JSON output format. The user prompt concatenated the target database name, the flattened schema representation, and the natural language question.

During generation, decoding was constrained to a limit of 256 with a repetition penalty of 1.1 to prevent looping artifacts. Because LLMs can occasionally generate conversational filler alongside structured data, our post-processing pipeline scans the raw string output for the first instance of bounded braces (...) and attempts extraction, failing to an empty dictionary if the output is malformed, ensuring evaluation scripts do not crash on generation errors. In our main inference, we remove invalid schemas with hallucinated table names or column names.

2.3 RapidFireAI Framework

During pipeline development, we encountered some bugs within the `rapidfireai` API that required runtime monkey-patching to ensure training stability and correctness.

- Grid Search Parameter Exclusion: The native `RFGGridSearch` implementation failed to iterate over arbitrary additional_kwarg when compiling run configurations. We patched `_get_runs_fit` with a recursive expansion function (`recursive_expand_gridsearch`) to properly unpack and inject additional keyword arguments into the leaf configurations. This makes it so no hyperparameters were ignored.

- **Missing Generation Prompts:** When preparing validation data in `_prepare_data`, the framework applied the tokenizer’s chat template but omitted the `add_generation_prompt=True` flag. This caused the model to treat the input as an unfinished user message rather than prompting it to begin the assistant’s response. We patched the method to explicitly append the generation prompt.
- **Left-Padding and Attention Masking Failures:** During batched generation, the framework failed to properly handle left-padded sequences. We resolved this by dynamically switching the tokenizer’s `padding_side` to "left" before generation. Furthermore, the original loop failed to pass the `attention_mask` to `model.generate()`. We updated the tokenization step to cache the `attention_mask` and patched `_run_generation_loop` to explicitly inject it into the generation arguments, ensuring the model correctly ignored pad tokens during inference.

Leveraging RapidFire AI’s multi-config API, we parallelized the evaluation of distinct hyperparameter combinations across five experimental blocks, each which may have multiple runs. This approach allowed us to isolate the impacts of LoRA dimensions, optimization learning rates, prompt engineering strategies, training durations, and base models.

2.4 Experimental Design

Here is the knob rationale we used:

- **LoRA Capacity Exploration:** We initially investigated the structural capacity of the low-rank adapters by co-varying the LoRA rank (r), scaling factor (α), and the target attention layers. We contrasted a localized adaptation strategy targeting only query and value projections (C_1, C_3) against an all-linear adaptation strategy targeting all projection layers (C_2, C_4). C_4 ($r = 32, \alpha = 64$, all target modules) yielded the highest initial evaluation score, establishing our baseline architecture.
- **Learning Rate Tuning:** Retaining the structural matrix from C_4 , we adjusted the learning rate knob to evaluate optimization stability. We evaluated a lower boundary of 1×10^{-4} (C_5) and an upper boundary of 4×10^{-4} (C_6) against the baseline 2×10^{-4} . Neither adjustment outperformed the baseline, leading us to use the original 2×10^{-4} learning rate.
- **Extreme Scaling Boundary:** To test if the task benefited from additional model capacity, we pushed the adaptation rank to an extreme setting ($r = 64, \alpha = 128$) in C_7 . This scaling did not produce downstream improvements, verifying that $r = 32$ offered sufficient representational capacity without overfitting.
- **Prompt Engineering and Overfitting Analysis:** Config C_8 introduced a JSON-constrained schema-linking system prompt and extended the training horizon to 10 epochs. This allowed us to monitor validation performance over time. We observed that overfitting began exactly at epoch 4, validating our choice of a 3-epoch training ceiling for prior configurations.
- **Base Model Ablation:** We introduced a final config (C_9) to test cross-model generalizability. Utilizing the exact same hyperparameters as C_8 , we substituted the base model for Qwen/Qwen2.5-0.5B-Instruct. This allowed us to evaluate whether the learned schema-linking capabilities scaled efficiently to a model with smaller parameters.

ID	Run	Base Model	r	α	Target Modules	LR	Prompt Type	Epochs
C_1	1	Llama-3.2-1B	8	16	[q_proj, v_proj]	2×10^{-4}	Baseline	3
C_2	2	Llama-3.2-1B	8	16	All Linear*	2×10^{-4}	Baseline	3
C_3	3	Llama-3.2-1B	32	64	[q_proj, v_proj]	2×10^{-4}	Baseline	3
C_4	4	Llama-3.2-1B	32	64	All Linear*	2×10^{-4}	Baseline	3
C_5	5	Llama-3.2-1B	32	64	All Linear*	1×10^{-4}	Baseline	3
C_6	6	Llama-3.2-1B	32	64	All Linear*	4×10^{-4}	Baseline	3
C_7	7	Llama-3.2-1B	64	128	All Linear*	2×10^{-4}	Baseline	3
C_8	8	Llama-3.2-1B	32	64	All Linear*	2×10^{-4}	Structured JSON	10
C_9	9	Qwen2.5-0.5B	32	64	All Linear*	2×10^{-4}	Structured JSON	10

Table 1: Summary of the 9 unique experimental config knobs. All Linear targets include [q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj]

2.5 Configuration

Table 1 summarizes the exact hyperparameter configurations utilized across all 9 unique experimental runs.

3 Results and Analysis

3.1 Configs

Our configs all showed a decrease in loss throughout our SFT. We can see an example epoch for a config in Figure 1.

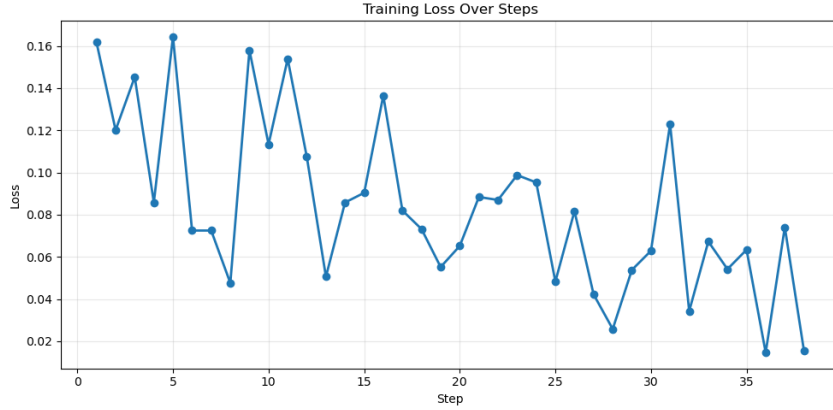


Figure 1: Loss decreases over time for C_6 for last epoch.

However, if we look at our evaluation of our 10 epoch config C_8 in Figure 2, we can observe overfitting at epoch 4. Interestingly, column recall increased on the 4th epoch, possibly because the model decided to return more columns overall to reduce loss and overfit.

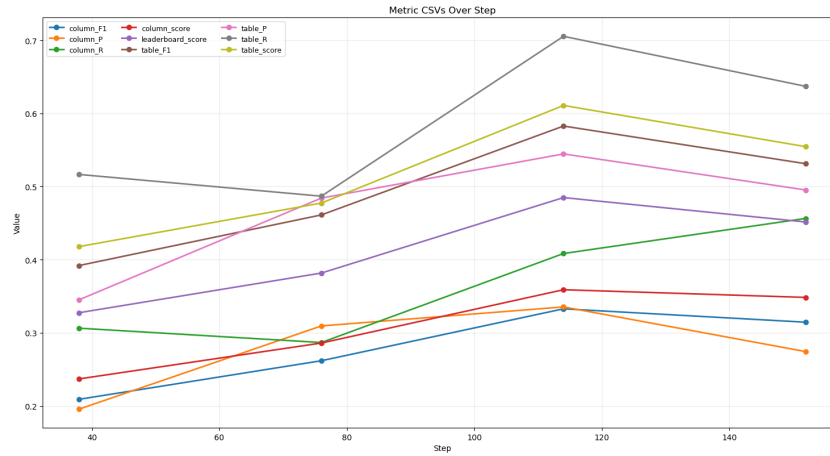


Figure 2: Overfitting for C_8 over 4 epochs. Note that C_8 stopped at epoch 4.

Our evals for all configs is in Table 2, showing table and column level metrics based on the hyperparam knobs we chose. Our best was clearly C_4 , which are the parameters we used for our final pipeline. All linear targets did better because schema linking is not just an attention/routing problem, it is also a translation and lexical mapping problem. By applying adapters to these MLP layers and the output projection the model has the capacity to memorize the specific mappings between natural language concepts and weird database identifiers. Additionally, having a sweet spot rank

Config	Table-Level			Column-Level			Leaderboard
	Precision	Recall	F ₁	Precision	Recall	F ₁	Score
Config 1	0.5117	0.6097	0.5281	0.2907	0.3437	0.2747	0.4264
Config 2	0.5784	0.6881	0.5983	0.4059	0.4268	0.3766	0.5124
Config 3	0.5186	0.6452	0.5441	0.3565	0.3880	0.3286	0.4635
Config 4	0.6309	0.6939	0.6314	0.3951	0.4409	0.3808	0.5288
Config 5	0.5545	0.6469	0.5633	0.3704	0.4070	0.3437	0.4809
Config 6	0.5459	0.6469	0.5660	0.3579	0.4219	0.3554	0.4824
Config 7	0.5833	0.6675	0.5920	0.3567	0.4174	0.3518	0.4948
Config 8	0.4953	0.6370	0.5312	0.2743	0.4564	0.3145	0.4515
Config 9	0.5990	0.6460	0.5960	0.4089	0.4293	0.3857	0.5109

Table 2: Hyperparameter search results across Configurations 1 through 9. Note that C_8 stopped at epoch 4.

capacity did impact score a lot too. The model type did not heavily impact score, possibly showing our comprehensive C_4 knobs determine evaluations the most rather than the model we used itself.

Additionally, we implemented a final pass that filters out hallucinated table names and column names, leading to our final evaluation in Table 3.

Granularity	Precision	Recall	F1 Score	Composite Score
Table-Level	0.6198	0.6964	0.6242	0.6468
Column-Level	0.4239	0.4445	0.3925	0.4203
Final Leaderboard Score:				0.5330

Table 3: Final evaluation. The model achieved exactly 0 schema-invalid identifiers across all generated outputs after our filtering pass.

3.2 Extra Things We Tried

We tried a fuzzy matching pipeline that normalizes table names, column names, and the user question. It then fuzzy matches between the question and the table names and column names. A score is assigned to each table based on the table’s name and its column’s names, tested against the question. There are hyperparameters to sorting based on the scores, choose a top K ratio, or filter with a score threshold. Modifying any of these parameters seemed to not improve with an unfiltered schema.

THRESHOLD	TOP_K	SORT	Table Score	Col Score	Leaderboard
0.01	0.9	True	0.581	0.386	0.483
0.05	1	True	0.59	0.381	0.486
0.1	0.5	True	0.431	0.276	0.354
0.05	1	False	0.623	0.391	0.507

Table 4: Tuning of hyperparams for fuzzing. This is only tested against the main.py run 4 final pipeline.

All of these impacted the evaluation negatively. We think this is because we did not train with this schema filtering, meaning our fine tuned model is expecting full schemas and databases without reordering. Our fine tuning is then thrown off when inference reorders and filters out schemas from databases that it got used to. If we had more time, we would like to train with schema filtering too. In addition, our fuzzy scores were very noisy when it came to table and column names that were entirely acronyms.

3.3 Extra Things We Want to Try

We wanted to try many things with more time and compute.

We wanted to try a decoupled agentic pipeline. Right now, the model executes a single-shot prediction, processing the entire schema context and attempting to identify exact table/column pairs simultaneously. Instead, we can have a model pass analyzing only the user query and a high-level catalog of table names alongside natural language descriptions to filter out the top K candidate tables.

Operating exclusively on the pruned schema subsets of the selected tables, we then can expand glossary definitions and use a fine-tuned adapter to pinpoint specific columns. Then a deterministic traversal script checks foreign key relationships. If dependent join tables are missing, they are automatically injected before final code compilation.

We also wanted to do a vector-search-based schema RAG. Table and column descriptions are mapped to vector spaces offline using an embedding model, similar to last project. At inference time, a hybrid query search (like vector similarity paired with BM25) selectively builds a localized mini-schema context, minimizing token degradation and resolving attention issues.

We would also like to implement data augmentation. We can even deliberately inject misleading "distractor" tables into training samples to train precision.

4 AI Tools Statement

We used LLM tools such as Claude and Gemini to help generate code that was mostly boilerplate. We consulted LLMs on ideas inspired by modern FT techniques, especially with SQL queries. We always made sure we fully understood the code and that it was correct.